



Cairo University
Faculty of Computing and Artificial Intelligence
Department of Computer Science



جَامِعَةُ الْقَاهِرَةِ
CAIRO UNIVERSITY

SmartGallery

Supervised by
Dr. Basheer Youssef
TA. Randa
ElBehery

Implemented by

<i>20220097</i>	<i>George Raafat Saber</i>
<i>20220309</i>	<i>Mohamed Mostafa Ali</i>
<i>20220208</i>	<i>Ali Elsayed Ali</i>
<i>20220360</i>	<i>Nour Eldeen Alaa</i>
<i>20220416</i>	<i>Youssef Nasser Mohamed</i>

Graduation Project
Academic Year 2025-2026
Midyear Short Documentation

Table Of Content

1.Abstract	3
2.Background	3
2.1.Main Area	3
2.2.Motivation	3
2.3.Beneficiaries	3
2.4.Main Techniques	3
2.5.Main Application	3
3.Problem Definition	4
4.Related work	4
5.Project specifications	5
5.1.System Architecture	6
5.2.Stakeholders	6
5.3.Functional Requirements	7
5.4.Non-Functional Requirements	7
5.5.Use-Case Diagram	8
5.6.Class Diagram	9
5.7.Sequence Diagrams	10
5.8.ERD	11
5.9.AI Models	11
6.Work Plan	12
7.References	15

List of figures

Figure 5.1. - System Architecture.	6
Figure 5.2. - Use Case Diagram	8
Figure 5.3. - Class Diagram	9
Figure 5.4. - Sequence Diagrams	9
Figure 5.5. - Sequence Diagrams	10
Figure 5.6. - ERD	10
Figure 6.1. - Gantt Chart	14

List Of Tables

Table 4.1. - Related Work.	5
Table 6.1. - Work Plan	12

1. Abstract

Modern smartphones have transformed into primary storage devices for personal memories and documents, leading to users accumulating thousands of images. However, retrieving specific photos from these massive collections is inefficient, as standard gallery applications rely heavily on basic metadata like timestamps or geolocation rather than visual content. While cloud-based solutions offer intelligent search, they compromise user privacy by requiring off-device data processing.

This project, **SmartGallery**, addresses this retrieval gap by developing a privacy-first, intelligent Android application. The proposed solution performs all image analysis locally on the device, ensuring user data never leaves the phone. The system utilizes a **CLIP-based** machine learning model to generate semantic embeddings for natural language search, integrated alongside **Optical Character Recognition (OCR)** for analyzing text in images, and **Face Recognition** for identifying individuals. By leveraging these on-device AI techniques, the project aims to provide a responsive, intuitive search experience that prioritizes both **efficiency and data privacy**.

2. Background

2.1. Main Area

The project lies at the intersection of Mobile Application Development and Artificial Intelligence, specifically focusing on Computer Vision and On-Device Machine Learning (Edge AI).

2.2. Motivation

The primary motivation is to resolve the conflict between utility and privacy. Users currently face a trade-off: they can either use "dumb" local galleries that respect privacy but offer poor search capabilities, or "smart" cloud galleries that offer excellent search but compromise data sovereignty. This project is motivated by the need to democratize AI-powered search features without relying on proprietary, closed-source cloud services.

2.3. Beneficiaries

The direct beneficiaries are Android smartphone users who manage large offline photo libraries and prioritize data privacy.

2.4. Main Techniques

The system employs **Vectorization** using a CLIP-based model to convert images into semantic embeddings. It utilizes **Cosine Similarity Calculation** to match user text queries with image vectors. Supporting techniques include **Face Encoding & Clustering** for people identification and **OCR** for text extraction.

2.5. Main Application

The final output is a native Android application developed in Kotlin that serves as a fully functional gallery alternative, featuring a "Smart Indexing Pipeline" that runs in the background to analyze new media

3. Problem Definition

The core problem this project addresses is the **Semantic Gap and privacy** in mobile image retrieval.

Smartphones store thousands of images, yet standard gallery applications limit users to linear chronological scrolling or basic metadata filtering. This interaction model is inefficient for retrieving content based on visual context (e.g., "picnic in the park") or specific contained text.

While AI-powered search exists in major commercial products, it predominantly relies on cloud computing architectures. This introduces significant issues:

- **Privacy Risks:** Personal photos must be uploaded to external servers for processing.
- **Dependency:** Features often break without an internet connection.
- **Latency:** Search speed is dependent on network conditions.

This presents a dilemma for the user:

- **Efficiency without Privacy:** Use cloud-based services that offer semantic search but require uploading personal data to remote servers, introducing privacy risks.
- **Privacy without Efficiency:** Use standard offline galleries that keep data secure but lack intelligence, forcing manual scrolling through thousands of files.

There is currently a lack of open-source & accessible Android solutions that provide state-of-the-art semantic search capabilities entirely on-device.

4. Related work

Several solutions exist in the domain of image management, yet they differ significantly in terms of **privacy architecture** and **processing location**.

- **Cloud-Based Ecosystems (e.g., Google Photos):** These represent the current industry standard for "smart" galleries. They offer advanced semantic search, face clustering, and object detection. However, they are **cloud-dependent** and **closed-source**. They require constant internet connectivity for full functionality and, critically, involve off-device data processing, which raises significant privacy concerns regarding user data ownership.
- **Standard Offline Galleries:** These applications run locally on the device, ensuring privacy and speed. However, their search capabilities are typically limited to **metadata** (date, location). They generally lack the deep learning integration required to understand the *semantic* content of an image (e.g., understanding that an image contains a "beach" without a manual tag), making them inefficient for large libraries.
- **Comparison & Differentiation: Smart Gallery** differentiates itself by combining the intelligence of cloud solutions with the privacy of local apps. The total comparison between our proposed solution and the different competitors is illustrated in *Table 4.1* below.

Table 4.1. - Related Work.

Comp.	Open source	Search for general prompts in photos	Search for general prompts in videos	Free to use	Face recognition	Must be online (privacy)	document Search
Smart Gallery	Yes (Project Goal)	Yes (On-Device CLIP/NLP)	No / Limited (Hardware constraints)	Yes (No Subs/Data Mining)	Yes (Local Embeddings)	No (Strict Offline)	Yes
Google Photos	No	Yes (Cloud-Based Multi-modal) / limited	Yes (Cloud Processing)	Freemium (Data Mining)	Yes (Cloud Clustering)	Yes (Cloud Dependence)	Yes
iPhone Gallery	No	Yes (On-Device Hybrid)	Limited	Yes (Hardware Cost)	Yes (On-Device)	Hybrid (Syncs Meta)	Yes
Samsung Gallery	No	Limited (Keyword/Tag based)	Limited	Yes (Hardware Cost)	Yes (On-Device)	Hybrid (Cloud Sync)	No

5. Project specifications

This section describes the overall design and internal workflow of the SmartGallery system, explaining how its components interact from data ingestion to intelligent search. It presents the architectural structure, system behaviour, and database organization through a set of supporting diagrams that illustrate the flow of information and the relationships between different modules.

Kotlin (Application Development):

The system is implemented as a native Android application using Kotlin. Kotlin was selected primarily for its high performance and direct access to Android system APIs without cross-platform abstraction layers found in frameworks such as Flutter, allowing finer control over background processing and efficient on-device AI execution.

SQLite (Local Data Management):

SQLite is used as the on-device database to store metadata, embeddings, and OCR results. It was chosen over ObjectBox because it is natively built into Android, has a smaller storage footprint, and provides built-in Full-Text Search (FTS) support, while ObjectBox introduces an additional third-party dependency. For the expected scale of up to 20,000 images, SQLite offers sufficient performance without requiring complex indexing methods such as Hierarchical Navigable Small World (HNSW). The database design remains simple and modular, allowing a future transition to ObjectBox as a potential optimization if scalability demands increase.

Open Neural Network Exchange (ONNX):

ONNX Runtime enables efficient on-device execution of AI models. ONNX was selected over alternatives such as ExecuTorch due to its higher maturity, better model portability across platforms, and stronger community support, which reduce stability risks and make debugging and maintenance more manageable during development.

5.1. System Architecture

The system consists of two main workflows: **image indexing** and **search execution**, as illustrated below in *Figure 5.1*. When a new image is detected, it passes through the indexing stage, where semantic embeddings, textual metadata, and face information are extracted and stored locally. During search, user queries from the mobile UI are processed by the search engine, where prompts are encoded and compared against stored embeddings using cosine similarity, while text-based queries are handled using Full-Text Search (FTS) to retrieve matching results. The system then returns ranked gallery results to the user interface.

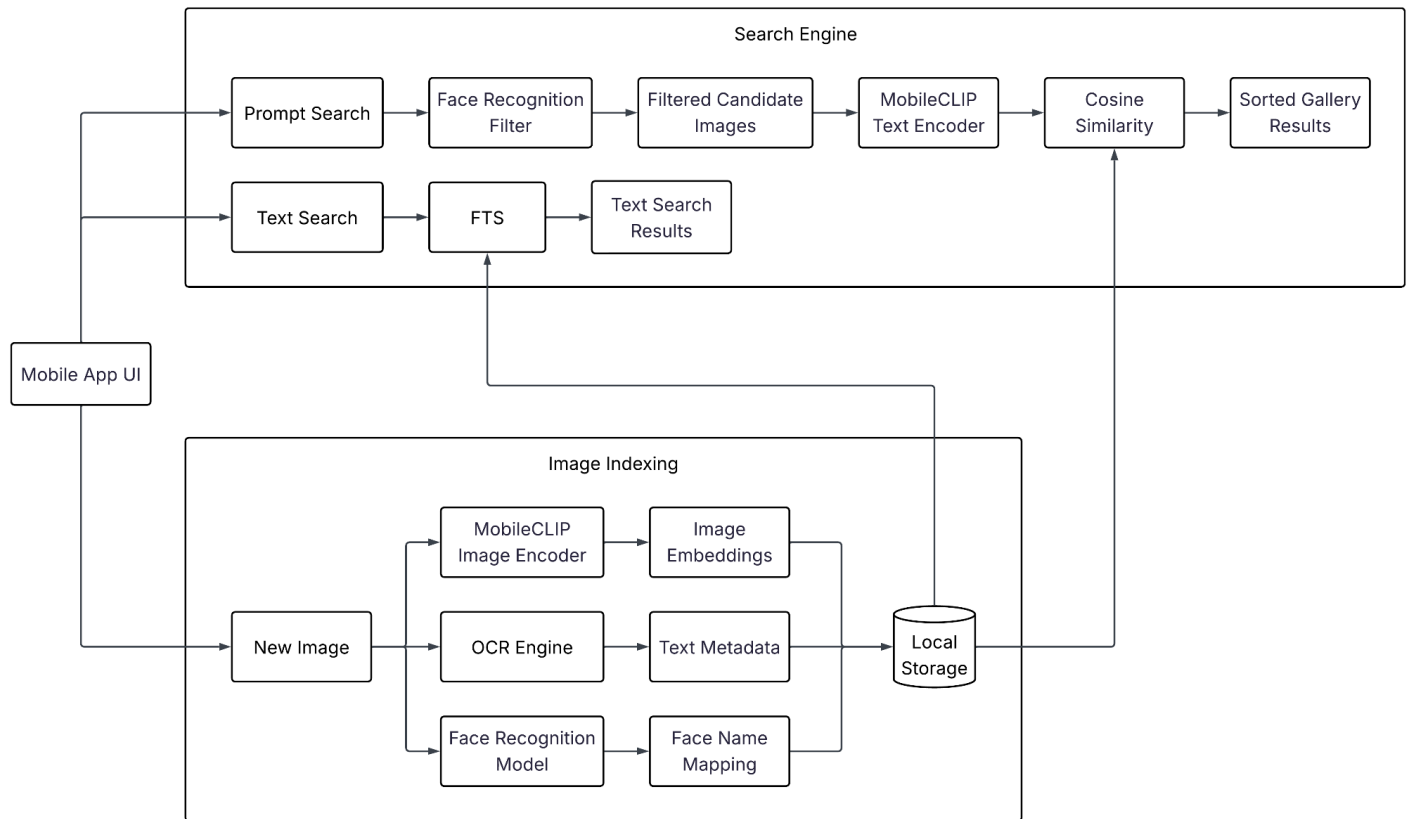


Figure 5.1 - System Architecture.

5.2. Stakeholders

- **Internal Stakeholders (Development Team):**

The team of developers and testers who worked and helped test the project.

- **External Stakeholders:**

- End users: The users who will use the program in their daily life. Examples: (photographer, anyone who has an Android device).
- AI Model providers: The app uses CLIP model from OpenAI and OCR model from Google.

5.3. Functional Requirements

- The system shall analyze images locally on the mobile device using an on-device vision-language model to generate semantic image embeddings.
- The system shall extract textual content from images using on-device OCR and associate the extracted text with the corresponding images.
- The system shall store image embeddings, extracted text, and metadata in a local database on the device.
- The system shall allow users to browse their image gallery and view stored images.
- The system shall support natural language search by comparing text query embeddings with stored image embeddings.
- The system shall retrieve and display images that are semantically similar to the user's search query.
- The system shall allow users to view detailed information for each image, including extracted text and associated metadata.
- The system shall support basic face labeling functionality for improved photo organization.
- The system shall operate fully offline for all core functionalities, including image analysis, OCR, and search.
- The system shall allow users to delete selected photos from the local gallery and remove all associated metadata, extracted text, and embeddings from local storage.
- The system shall allow users to share selected photos with other applications using the device's native sharing mechanisms, without exposing internal analysis data.

5.4. Non-Functional Requirements

- The total application size shall not exceed 500 MB to ensure efficient storage and execution on mobile devices.
- The system shall perform image embedding inference within ≤ 1 seconds per image on mid-range Android devices, enabling efficient image indexing and semantic search.
- The system shall return image search results within ≤ 1 second for local image collections of up to 10,000 images.
- The system shall scale to support at least 20,000 images, with no more than 20% degradation in search performance compared to baseline operation.

- The system architecture shall be modular and maintainable, allowing future replacement or upgrading of machine learning models without affecting other system components.
- The system shall provide an intuitive and user-friendly interface, enabling users to complete common tasks such as image search within three or fewer user interactions.
- The system shall be primarily optimized for Android devices (Android 10 and above), while supporting multiple screen sizes and device configurations.

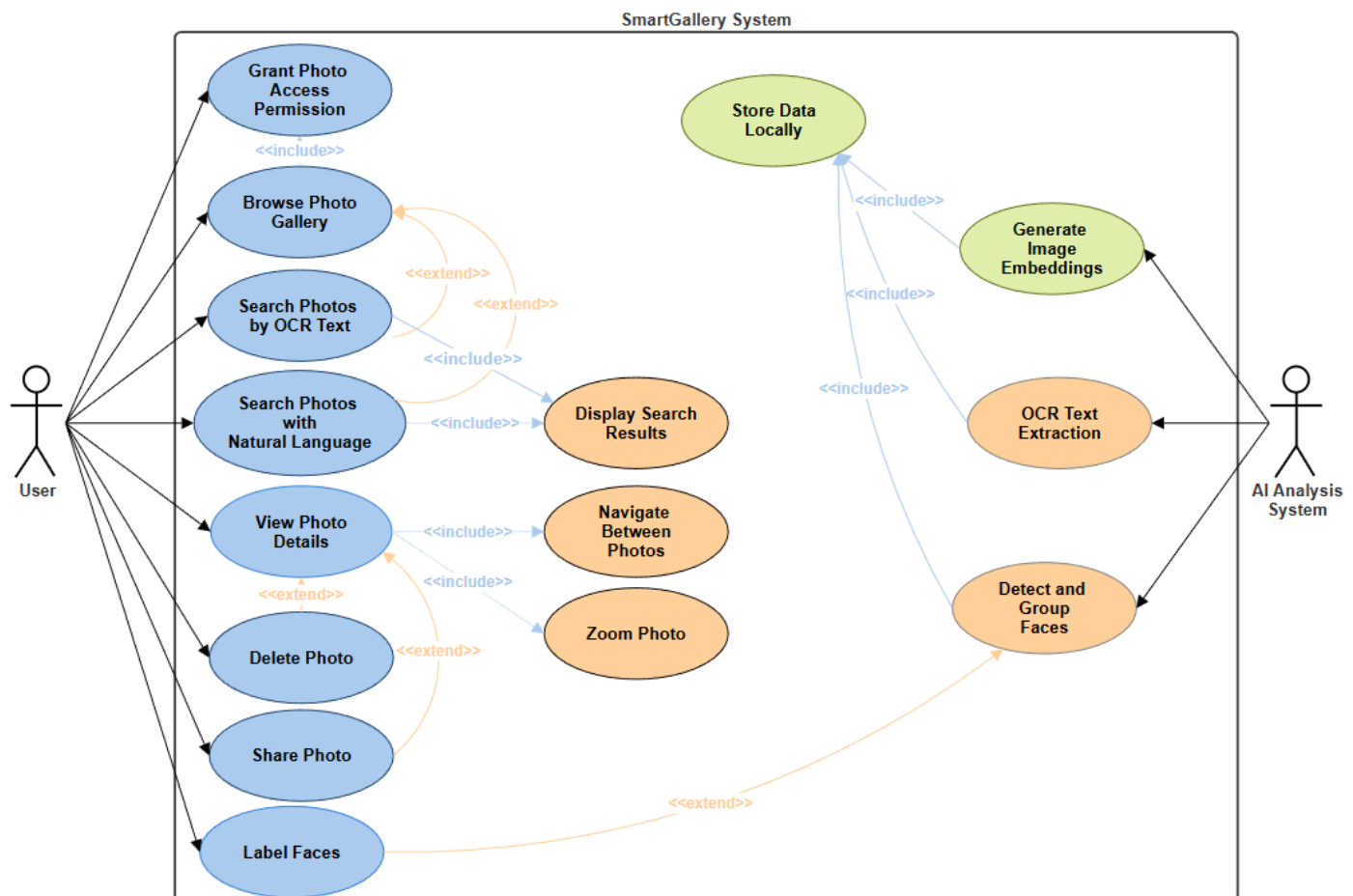
5.5. Use-Case Diagram

This section presents the Use Case Diagram as illustrated below in *Figure 5.2.* for the SmartGallery system, illustrating the main interactions between users and the system and highlighting the core functionalities provided by the application.

Each use case represents a user-initiated action or a system behavior that delivers a specific value to the user.

The use case diagram is derived directly from the functional requirements and serves as a high-level behavioral view of the system, without addressing implementation details.

Yes (Hardware Cost) Yes (Hardware Cost)



5.6. Class Diagram

The class diagram illustrates the main structural components of the Smart Gallery system and the relationships between core modules, as illustrated below in *Figure 5.3*. It represents how data models, processing components, and the application layer interact to support image indexing, search functionality, and overall system organization.

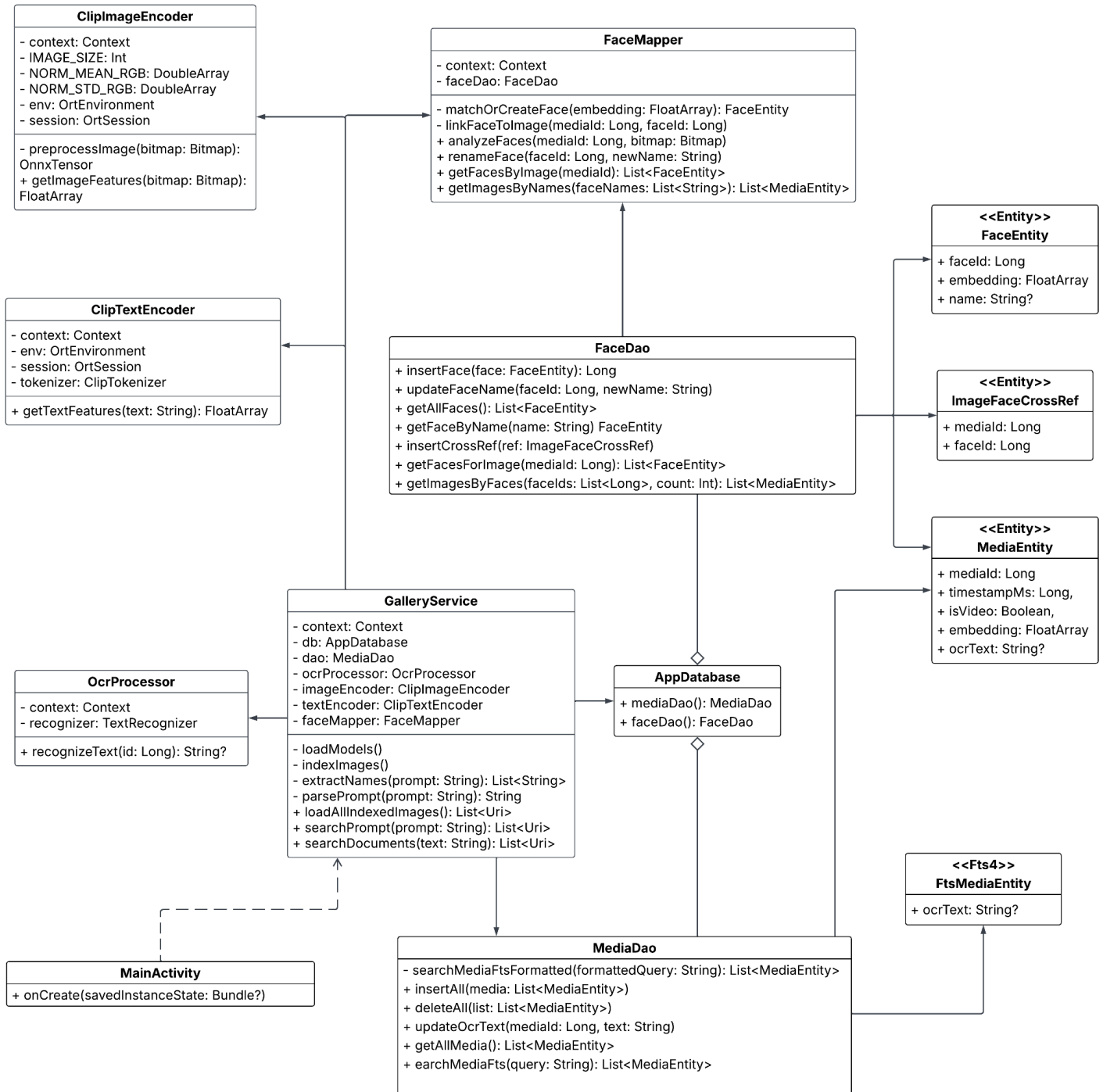


Figure 5.3. - Class Diagram

5.7. Sequence Diagrams

The following sequence diagram illustrated in *Figure 5.4*. shows the semantic search pipeline powered by the CLIP (Contrastive Language-Image Pre-training) model. This process describes how a user's natural language prompt is converted into a high-dimensional vector (embedding) and then compared against the pre-calculated embeddings of the gallery images.

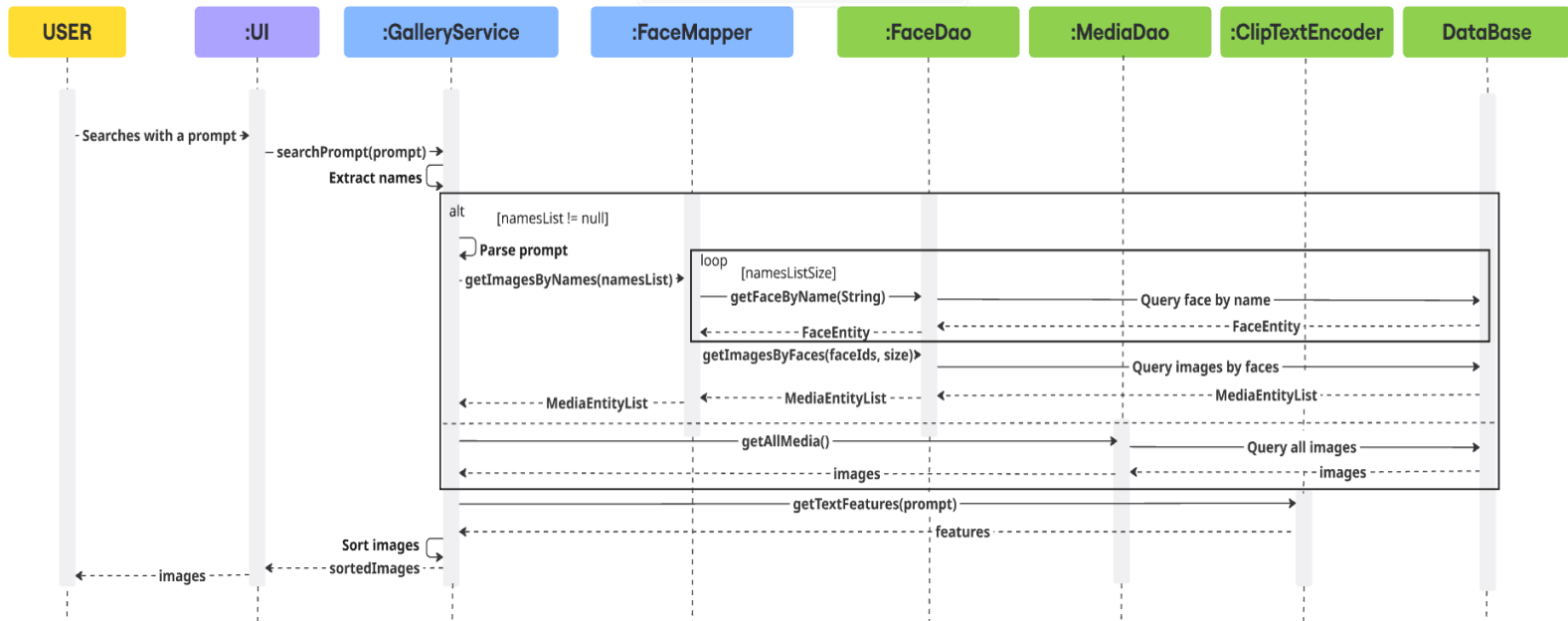


Figure 5.4. - Prompt Search using CLIP Sequence Diagram

In contrast to semantic searching, the OCR search pipeline illustrated in *Figure 5.5*. focuses on precise textual retrieval. The process describes how the text search gets handled in the system.

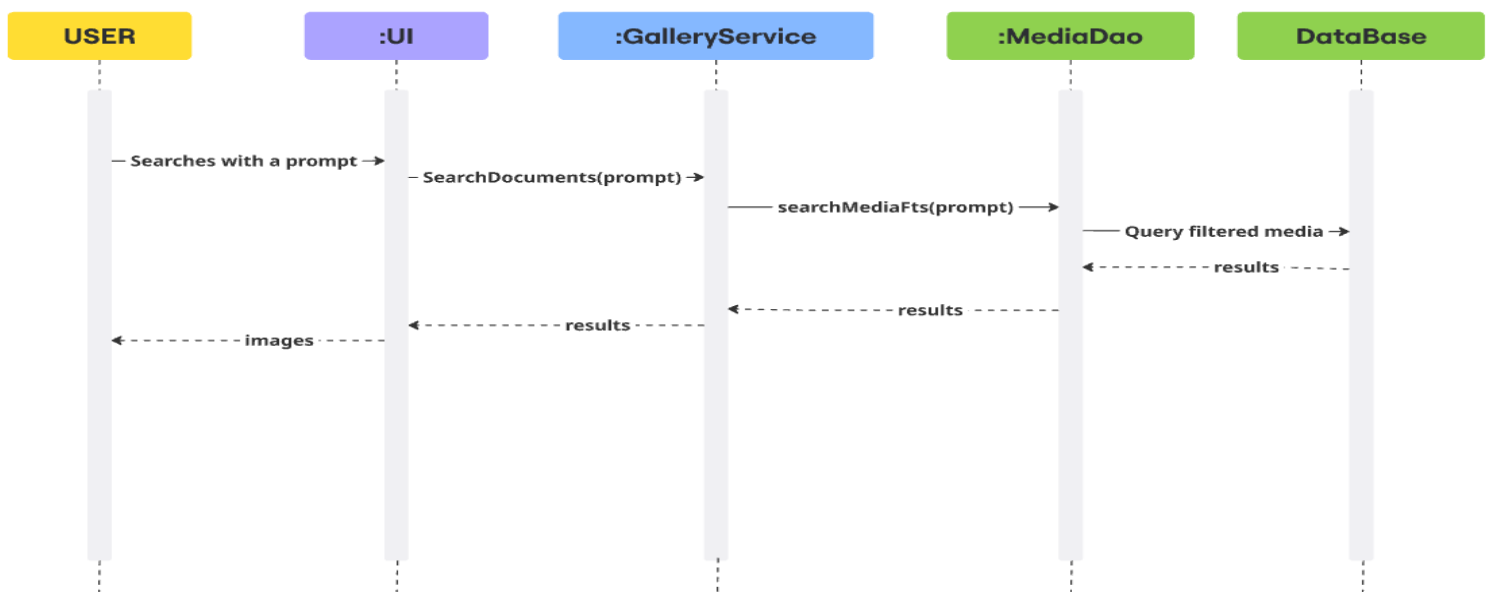


Figure 5.5. - Text Search using OCR Sequence Diagram

5.8. ERD

To support the core functionality of the SmartGallery, as illustrated below in *Figure 5.6.*, our database schema is designed to efficiently link physical image files with their extracted metadata and OCR content. The following Entity Relationship Diagram (ERD) illustrates our relational approach, highlighting the central role of the **FTS Virtual Table**.

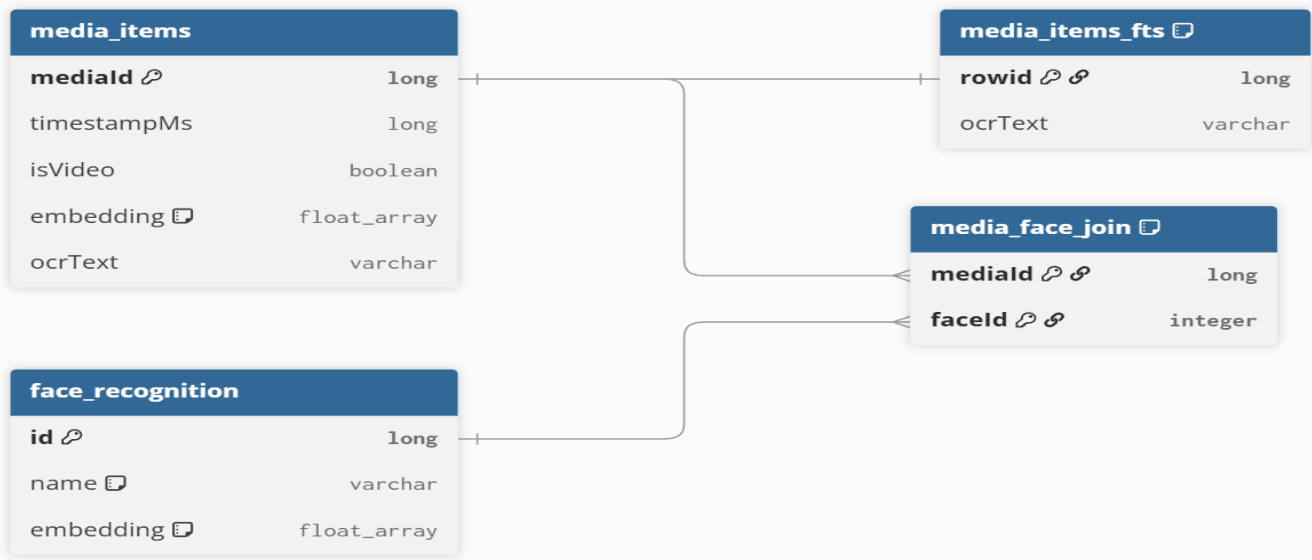


Figure 5.6. - ERD

5.9. AI Models

▪ CLIP:

The SmartGallery application uses the **MobileCLIP2-S0** model to enable semantic image retrieval. MobileCLIP2-S0 is a lightweight vision-language model designed for on-device inference. Instead of training a custom model, the system relies on a **pre-trained checkpoint**, allowing efficient embedding generation without additional training.

The model was pre-trained on the **DFNDR2B dataset**, a large-scale multimodal dataset containing image-text pairs. The pre-training objective aligns image and text representations into a shared embedding space, enabling similarity comparison between user prompts and stored images.

Feature Representation:

○ Image Features:

Images are converted into numerical tensors before being passed into the MobileCLIP image encoder.

The input feature representation consists of $(3 \times 256 \times 256)$ tensor

- **3 channels** represent RGB color information.
- Each image is resized and center-cropped to **256×256 pixels**.

The output of the image encoder is a 512-dimensional embedding vector used for similarity search.

○ **Text Features:**

User prompts are processed using the MobileCLIP text encoder. Tokenization using a CLIP tokenizer. Padding or truncation to a fixed sequence length of 77 tokens.

▪ **Face Recognition Features (Planned Component):**

Because the final model has not yet been selected, the exact feature representation is currently unspecified. However, the planned approach follows a standard embedding-based recognition pipeline:

- detected faces will be converted into embedding vectors
- Similarity comparison will be used to match identities
- embeddings will be stored locally to allow incremental identity learning

6. Work Plan

This section presents the structured development timeline of the Smart Gallery project, outlining the main phases and milestones achieved throughout the implementation process, as summarized in *Table 6.1*. The work plan spans from **August 2025 to June 2026**. The listed tasks reflect a phased and iterative workflow that guided the project's technical progress and ensured continuous evaluation of development goals.

Table 6.1. - Work Plan

Task	Task Title	Description	Task status
1	Brainstorming	Explore multiple project concepts through brainstorming and initial research, systematically evaluating and discarding less viable ideas to arrive at the final project direction.	Completed
2	Requirements Analysis	Define the main system goals, including semantic search, OCR-based retrieval, and identity recognition features.	Completed
3	Feasibility study	Investigate technical requirements, on-device AI	Completed

		capabilities, and possible implementation approaches	
4	Search for related work	Find competitors and know the strengths and weaknesses of their features	Completed
5	Technology stack evaluation	Analyze different mobile development frameworks and assess their suitability for performance and usability	Completed
6	Model usage research	Learn how vision-language models are integrated, executed locally, and used for embedding generation	Completed
7	Model comparison study	Review multiple CLIP or vision-language model variants based on size, speed, and accuracy	Completed
8	Inference framework comparison	Compare mobile inference solutions such as ONNX Runtime and ExecuteTorch in terms of performance and compatibility	Completed
9	System architecture design	Design the overall workflow, including embedding generation, indexing, search pipeline, and data handling	Completed
10	Learning Kotlin	Study the Kotlin programming language and understand best practices for Android development	In progress
11	Diagrams	UML, ERD, Use case, and Sequence diagram	Completed
12	OCR integration	Implement text extraction from images to support keyword-based searching	Completed
13	Embedding & search implementation	Develop the core functionality for embedding images, queries, and ranking results based on similarity	In progress
14	UI/UX implementation	Develop and refine the user interface for browsing, searching, and interacting with the gallery	In progress
15	Face recognition feature	Integrate face detection and recognition capabilities into the search pipeline	Planned

16	Model optimization research	Investigate optimization techniques such as quantization and model compression	Planned
17	Performance optimization	Improve the efficiency of model inference, memory usage, and processing speed on mobile devices	Planned
18	Testing & evaluation	Conduct performance testing, usability evaluation, and validation of search quality	Planned
19	Final documentation	Prepare final technical documentation and summarize project outcomes	Planned

The following Gantt chart shows the progression and overlap of project phases throughout the development lifecycle, as illustrated below in *Figure 6.1*.

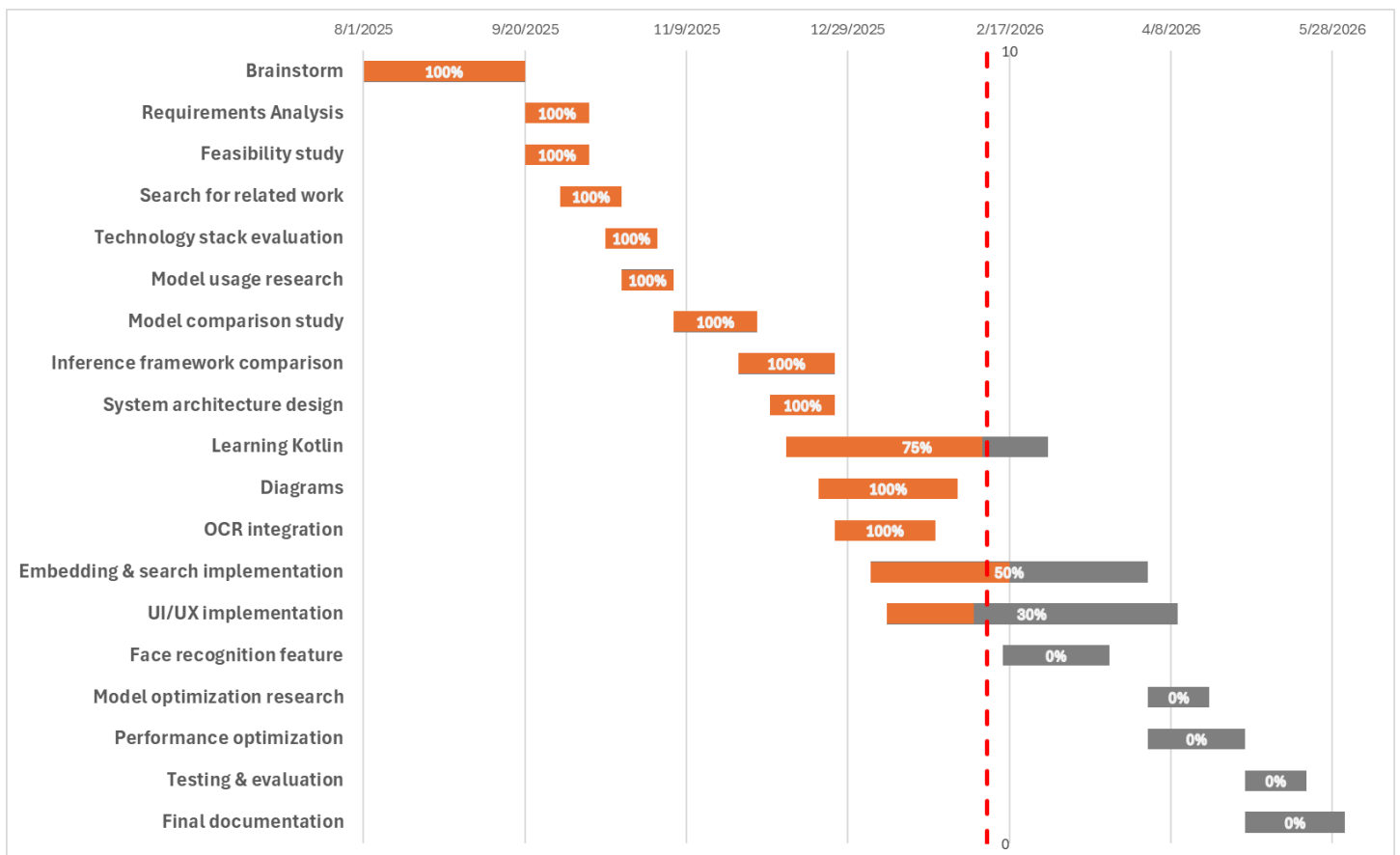


Figure 6.1. - Gantt Chart

7. References

- [1] mlfoundations, “open_clip”, Github, 2025. [Online]. Available: https://github.com/mlfoundations/open_clip
- [2] apple, “ml-mobileclip”, Github, 2025. [Online]. Available: <https://github.com/apple/ml-mobileclip>
- [3] wkcn, “TinyCLIP”, Github, 2025. [Online]. Available: <https://github.com/wkcn/TinyCLIP>
- [4] F. Faghri et al., "MobileCLIP2: Improving Multi-Modal Reinforced Training" Apple, 2025. [Online]. Available: <https://arxiv.org/pdf/2508.20691>
- [5] A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision," in Proc. International Conference on Machine Learning (ICML), 2021. [Online]. Available: <https://arxiv.org/pdf/2103.00020>
- [6] ONNX, "Open Neural Network Exchange (ONNX)," 2025. [Online]. Available: <https://onnx.ai/>
- [7] PyTorch, “ExecuTorch,” 2025. [Online]. Available: <https://docs.pytorch.org/executorch/stable/index.html>
- [8] JetBrains, "Kotlin Programming Language," 2025. [Online]. Available: <https://kotlinlang.org/>
- [9] K. Wu et al., "TinyCLIP: CLIP Distillation via Affinity Mimicking and Weight Inheritance," 2023. [Online]. Available: <https://arxiv.org/pdf/2309.12314>
- [10] Flutter, “Flutter – Build apps for any screen,” 2025. [Online]. Available: <https://flutter.dev/>
- [11] X. Huang et al., "Tag2Text: Guiding Vision-Language Model via Image Tagging," 2024. [Online]. Available: <https://arxiv.org/pdf/2303.05657>
- [12] L. Yuan et al., "VOLO: Vision Outlooker for Visual Recognition," 2021. [Online]. Available: <https://arxiv.org/pdf/2106.13112>
- [13] Y. Zhang et al., "Recognize Anything: A Strong Image Tagging Model," 2023. [Online]. Available: <https://arxiv.org/pdf/2306.03514>